

TurboQuant in the Browser: A Feasibility Study of KV-Cache Compression for Gemma 4 on Chrome WebGPU

Dhonam Pemba Stratos Lab — dhonam@stratoslab.xyz

Kwang Wei Sim Stratos Lab — kwang@stratoslab.xyz

Abstract

KV-cache growth is a central bottleneck for long-context autoregressive inference, and the challenge intensifies in browser environments where GPU memory budgets, CPU-to-GPU transfer overhead, and runtime abstraction costs impose additional constraints.

TurboQuant has been proposed as a training-free low-bit vector quantization method capable of compressing LLM KV caches to approximately 3 bits on H100-class hardware. This paper investigates whether a TurboQuant-inspired compression strategy can operate usefully within a browser inference stack built on transformers.js and Chrome WebGPU.

We implement an experimental TurboQuantCache in a fork of transformers.js, exposing it through the standard generation API, and benchmark it against a dense baseline on onnx-community/gemma-4-E2B-it-ONNX across five prompt categories and three quantizer configurations. The compressed path achieves 1.29–1.34× average KV-cache size reduction and 69.9–83.7% average prefix agreement with dense-baseline outputs. However, it is slower than the dense baseline under all tested conditions, with average end-to-end speed ratios of 0.50–0.57× and average decode throughput falling from 18.5 tokens/s to approximately 9–10 tokens/s. On the longest tested context, end-to-end decode throughput collapses to 0.74 tokens/s versus 9.98 tokens/s for the dense baseline, a 13.5× TPS gap that scales with KV-cache size. No configuration produces an exact-match output on any test case.

The central systems finding is that a cache-wrapper-only implementation — in which compressed KV state must be fully rematerialized into dense tensors before each ONNX decoder call — is not sufficient to achieve a latency benefit in this stack. The paper contributes a working transformers.js fork with a pluggable cache abstraction, a deterministic browser benchmark harness for Gemma 4 on Chrome WebGPU, and a quantitative characterisation of how rematerialization overhead scales with cache size.

Keywords: browser inference, WebGPU, transformers.js, KV cache, quantization, TurboQuant, Gemma 4, ONNX Runtime Web, LLM systems

1. Introduction

Browser-hosted LLM inference has become a viable deployment target. The combination of transformers.js, ONNX Runtime Web, and the WebGPU API now allows medium-scale

generative models to run directly in Chrome-class environments without server infrastructure, trading centralized deployment cost for lower latency for the end user, stronger privacy guarantees, and offline capability. However, the resource budget for browser inference remains fundamentally tighter than for datacenter deployments. GPU memory, bandwidth, and compute are all more constrained, and the software stack introduces API boundaries and data-movement costs that do not exist in native accelerator deployments.

One of the most acute pressure points is the autoregressive KV cache. In long-context decoding, the KV cache grows with every generated token, consuming an increasing fraction of available GPU memory and imposing mounting per-step costs for cache reading, writing, and management. For browser inference, this problem is compounded: the ONNX Runtime Web execution model defaults to CPU-resident tensors, and keeping the cache GPU-resident across decode steps requires explicit IO binding. Without it, each decode step may incur a full CPU-GPU round trip for the KV state.

TurboQuant [1, 2] has been presented as a training-free solution to KV-cache pressure. Its public description is attractive: compress KV caches to roughly 3 bits using a two-stage PolarQuant + residual-correction design, preserve model quality, and achieve large speedups on accelerator-native attention kernels. These results, however, are reported for an implementation environment — H100-class GPU hardware with custom attention kernels — that is very different from a browser stack running ONNX models through WebGPU.

This paper addresses a narrower question: *can a TurboQuant-inspired cache path be integrated into transformers.js-based browser inference, and what does it cost?* Our goal is not to reproduce TurboQuant’s accelerator-side claims. Instead, we build and evaluate a browser-realistic implementation — one that must work within the constraints of an ONNX model interface that expects dense past-key-value tensors — and report what happens.

The main result is negative on latency and mixed on quality. The implementation runs end-to-end in Chrome WebGPU, achieves real but modest KV-cache compression, and reveals a clear systems bottleneck: the per-step cost of rematerializing dense tensors from a compressed store scales roughly with the size of the KV cache and, at large context lengths, dominates the decode budget entirely.

Our contributions are:

1. **A transformers.js fork** that adds a pluggable cache abstraction (PastKeyValues) and an experimental TurboQuantCache with low-bit packed storage, optional rotation preprocessing, key residual correction, and a dense residual window (Section 3).
2. **A browser benchmark harness** for Gemma 4 on Chrome WebGPU that reports end-to-end latency, time-to-first-token (TTFT), decode throughput, packed and dense KV-cache byte counts, exact match, and prefix agreement between dense-cache and compressed-cache outputs (Section 5).
3. **An empirical analysis** demonstrating that the dominant performance cost in the current design is per-step dense rematerialization whose overhead scales with cached

sequence length, and that this cost is not offset by the compression gains at any tested operating point (Sections 6–7).

2. Background

2.1 KV-cache memory cost

For a decoder-only transformer, the dense KV-cache memory can be approximated as

$$M_{\text{dense}} \approx 2 \cdot L \cdot H_{kv} \cdot T \cdot D \cdot b,$$

where L is the number of transformer layers, H_{kv} is the number of key-value heads, T is the cached sequence length, D is the head dimension, and b is bytes per element. The factor of 2 accounts for separate key and value caches. For browser inference, the effective cost is larger than M_{dense} alone because the runtime also pays for tensor allocation, explicit CPU-GPU data transfers, and model-boundary tensor materialization at each decode step.

2.2 TurboQuant: algorithm and accelerator claims

TurboQuant [1, 2] is publicly described as a training-free, two-stage vector quantization method. Stage 1 (PolarQuant) applies a rotation or reparameterization to KV vectors that makes them more amenable to very-low-bitrate quantization. Stage 2 applies a 1-bit residual correction (QJL-style) to reduce bias in dot-product estimation after quantization. The combined system is reported to compress KV caches to approximately 3 bits while maintaining downstream task quality, and to accelerate attention-time computation on H100-class hardware.

These claims rest on an implementation context — native GPU kernels that consume the compressed representation directly without intermediate dense reconstruction — that does not transfer automatically to a browser stack. Our work tests how far the algorithmic idea can be taken within the constraints of the transformers.js + ONNX Runtime Web environment.

2.3 The browser inference stack and its constraints

transformers.js [4] runs ONNX models in-browser using ONNX Runtime Web, with device: 'webgpu' enabling GPU acceleration. The ONNX Runtime WebGPU guide [6] notes a detail that is critical for KV-cache work: tensor inputs and outputs default to CPU memory residency. Keeping tensors on the GPU across decode steps requires explicit IO binding with preferredOutputLocation: 'gpu-buffer'. Without IO binding, every decode step that reads prior KV state from CPU and writes new KV state to CPU incurs a full round-trip data movement cost.

This creates a fundamental tension for any cache compression scheme implemented as a wrapper around the standard ONNX generation path. If a compressed cache is stored on the CPU between decode steps but the ONNX session still expects full dense `past_key_values.*` tensors as inputs, then the system pays:

- CPU-side pack and unpack operations;
- dense tensor reconstruction before each decoder call;
- CPU-to-GPU transfer of the reconstructed tensors;
- GPU-to-CPU transfer of the `present.*` outputs.

Each of these costs scales with the size of the KV cache. Whether compression savings outweigh these overheads depends on how large the cache is relative to the reconstruction cost at a given sequence length. The results in Section 6 show that, at the context lengths tested, they do not.

The ONNX Runtime GenAI configuration reference [7] documents `past_present_share_buffer` as a generation-oriented optimization that allows key-value state to be shared across steps without repeated allocation. This is not the path used in the current work, but it is a relevant reference point for the GPU-resident cache designs discussed in Section 9.

3. System Design

3.1 Design objective

The goal of the fork was to introduce a cache implementation seam into `transformers.js` without rewriting the full generation pipeline. The design needed to satisfy two constraints simultaneously: the modified generation API must remain backward-compatible with the dense path, and both paths must be testable through the same benchmark harness on the same model and prompts.

3.2 Cache abstraction

The fork adds a `PastKeyValues` contract in `packages/transformers/src/cache_utils.js` with three core methods:

- `update(decoderResults, options)` — ingest and store new KV tensors from a decode step;
- `materialize(decoderFeeds)` — reconstruct the tensors required by the next ONNX decoder call;
- `getStats()` — return packed and dense byte counts for benchmark reporting.

This changes generation from a flat-map assumption about `past_key_values` to a cache-object-owned strategy. The decoder emits standard `present.*` tensors; the cache object decides how to store and reconstruct them.

Two concrete implementations are provided:

- `DynamicCache`: the original dense path, storing `past_key_values.*` as plain tensors.
- `TurboQuantCache`: the experimental compressed path described below.

The active implementation is selected via a `cache_implementation` generation option ("dynamic" or "turboquant"). Cache statistics are returned when `return_dict_in_generate: true` is set.

3.3 TurboQuantCache: what is implemented

The current TurboQuantCache includes:

- **Low-bit packed storage** for keys and values, with configurable bit widths b_k and b_v per tensor element;
- **Optional Hadamard-style rotation** when the head dimension is a power of two, applied as a preprocessing step before quantization;
- **Key residual correction**, which stores a residual to partially compensate for quantization error in the key vectors;
- **A dense residual window** of length T_r , keeping the most recent T_r tokens in full-precision storage to protect output quality at the boundary of the compressed region;
- **Cache-size reporting** through `packed_bytes` (the total compressed footprint) and `dense_bytes` (the equivalent uncompressed size).

3.4 What is not yet implemented

The current implementation is a TurboQuant-*inspired* approximation. It does not include:

- a fully faithful PolarQuant rotation stage as specified in the paper;
- a full QJL residual estimator;
- compressed-attention kernels that consume the packed representation directly;
- GPU-native compressed cache storage;
- any use of ONNX Runtime Web IO binding or GPU-buffer outputs.

The correct description is a *TurboQuant-inspired browser cache path*, not a full reproduction.

3.5 The materialization loop and its cost

The central architectural compromise is that the ONNX decoder still expects dense `past_key_values.*` inputs. The update–materialize loop at each decode step is therefore:

1. ONNX decoder produces `present.*` tensors (GPU → CPU transfer under default settings).
2. `TurboQuantCache.update()` ingests and packs the tensors on CPU.
3. `TurboQuantCache.materialize()` reconstructs dense `past_key_values.*` tensors on CPU.
4. Dense tensors are passed to the next ONNX decoder call (CPU → GPU transfer).

Steps 2–3 add pack-and-unpack CPU work at every decode step. Steps 1 and 4 are data-movement costs that also occur in the dense path, but any compression benefit in step 2 is lost when the full dense tensors must be reconstructed in step 3. This design makes the

experiment feasible without changing the ONNX graph interface, but it also accounts for the performance results in Section 6.

4. Metrics

We report five primary comparative metrics for each benchmark configuration.

4.1 Speed ratio

$$S = \frac{t_{\text{dynamic}}}{t_{\text{turbo}}},$$

where t_{dynamic} is the average end-to-end generation latency for the dense cache and t_{turbo} is the average for the compressed cache. $S > 1$ would indicate the compressed path is faster; $S < 1$ means it is slower.

4.2 Compression ratio

$$C = \frac{B_{\text{dense}}}{B_{\text{packed}}},$$

where B_{dense} is the dense-equivalent KV-cache footprint in bytes and B_{packed} is the packed cache footprint reported by `TurboQuantCache.getStats()`. $C > 1$ indicates genuine compression.

4.3 Prefix agreement

Let y_d be the dense-baseline output string and y_t the TurboQuant output string. Let $LCP(y_d, y_t)$ denote the length of their longest common prefix. Then

$$P = \frac{LCP(y_d, y_t)}{\max(|y_d|, 1)}.$$

This is a lightweight textual-agreement proxy, not a semantic correctness metric. A value of 1.0 means the outputs agree character-for-character up to the length of the dense output; lower values indicate divergence at some point. Prefix agreement can be high even when outputs differ significantly after the agreement point, and it does not capture rearrangement or paraphrasing.

4.4 Time to first token (TTFT)

TTFT is the elapsed time from the start of a generation call to the production of the first output token. It is dominated by the prefill (prompt encoding) pass for long inputs. We report it separately from end-to-end latency because it is less sensitive to per-step reconstruction overhead.

4.5 Decode throughput

Decode throughput is reported in tokens per second (TPS) as measured over the full generation call. Because TTFT is included in the denominator when computing overall TPS from the average latency, the reported TPS values combine prefill and decode costs. This means TPS comparisons between dense and compressed paths are sensitive to TTFT differences, particularly for long-context prompts where TTFT dominates.

4.6 Approximate compressed-cache size model

The theoretical packed footprint of the two-region cache is

$$M_{\text{turbo}} \approx 2 \cdot L \cdot H_{kv} \cdot [T_r \cdot D \cdot b_{\text{fp}} + (T - T_r) \cdot D \cdot b_{\text{eff}}],$$

where T_r is the dense residual-window length, b_{fp} is the per-element cost in the dense region, and b_{eff} is the effective packed bitrate in the compressed region. Larger T_r protects quality but reduces compression; smaller T_r increases compression but risks output drift.

5. Experimental Setup

5.1 Model and runtime

All experiments use:

- **Model:** onnx-community/gemma-4-E2B-it-ONNX
- **Model dtype:** q4f16
- **Runtime:** forked transformers.js (fork at <https://github.com/stratoslab/transformers.js>)
- **Execution environment:** Chrome WebGPU
- **Generation:** deterministic (do_sample: false)

The benchmark worker loads the forked transformers.js browser bundle directly, not the npm release package.

5.2 Prompt suite

Five prompt categories exercise different text-generation characteristics:

ID	Label	Description	Max tokens
risk-short	Risk Summary	Short enterprise risk review	48
ops-checklist	Operations Checklist	Structured compliance checklist	72
policy-com	Policy Comparison	Comparative reasoning task	80

pare			
long-cont ext-1 k	Long Context 1x	Extended prompt, cache stress	96
long-cont ext-2 k	Long Context 2x	Very long prompt, heavy cache pressure	96

The first three cases are short structured enterprise prompts. The latter two stress cache behavior under larger input sequence lengths.

5.3 Quantizer configurations

Three TurboQuant operating points are evaluated, parameterized by key bit width b_k , value bit width b_v , and residual window length T_r :

Configuration	b_k	b_v	T_r
Safe Default	4	8	64
Key Heavy	3	8	64
Mid Compression	4	8	48

Each TurboQuant configuration is compared against a common dense-cache baseline (dynamic) under identical model, prompt, and generation settings.

5.4 Run budget and reproducibility

Each configuration–case pair was run **2 times**, and the reported metrics are averages over those runs. Two runs is a minimal statistical budget; results should be treated as point estimates rather than robust statistics. Section 8 discusses this limitation explicitly.

The benchmark harness, fork commit history, and exported benchmark data are described in Section 10 (Data Availability). Specific fork commits relevant to the TurboQuant implementation are: db06d73, 4fab2e0, 24c4c0b, dfbf815, 0424f44, fa534d7, 3e268cc, 0b2b2d6, 529939a, and 337eb60. The benchmark harness application commit is 7af609f.

Hardware configuration was not systematically recorded in the current experiment. The software configuration (Chrome, WebGPU, q4f16 model dtype) is reproducible from the fork and harness.

6. Results

6.1 Aggregate configuration summary

Table 1 summarises performance averaged across all five benchmark cases.

Table 1: Aggregate performance by configuration (dense baseline: 1.000×, 18.5 TPS)

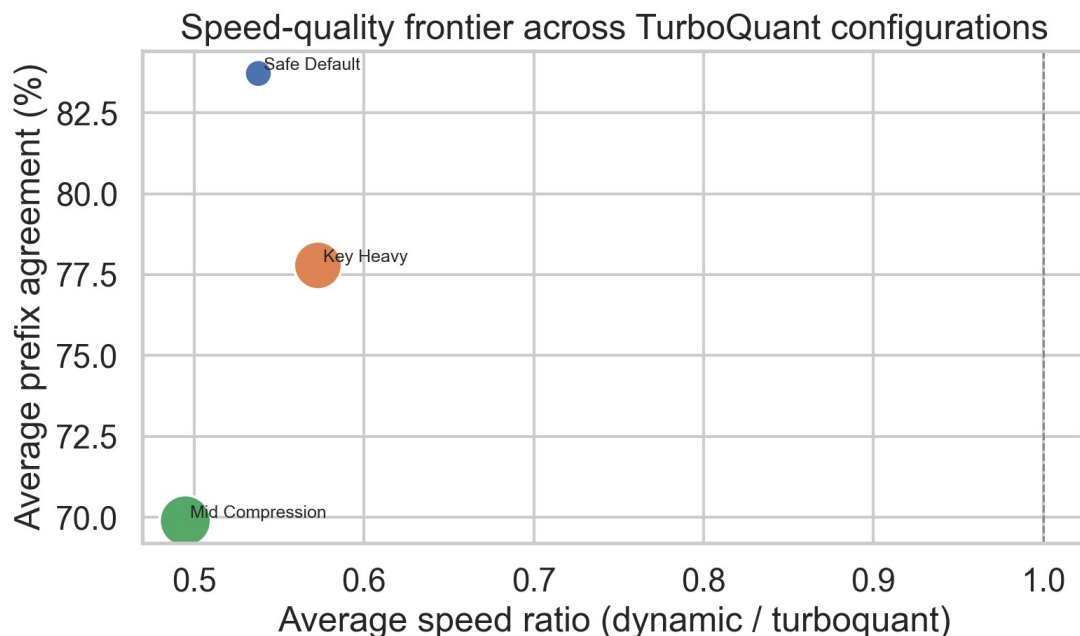
Configuration	Speed ratio	Compression	Prefix agr.	TPS (turbo)	Exact (n=5)
Safe Default	0.538×	1.290×	83.7%	10.0	0/5
Key Heavy	0.573×	1.328×	77.8%	10.1	0/5
Mid Compression	0.495×	1.336×	69.9%	9.1	0/5

All three configurations remain slower than the dense baseline. The best aggregate speed ratio is 0.573× (Key Heavy), meaning the compressed path takes roughly 1.75× longer end-to-end than the dense baseline on average. Average decode throughput across configurations is approximately 9–10 tokens/s, compared to 18.5 tokens/s for the dense path — roughly a halving of throughput. No configuration produces exact-match output on any of the five cases.

Compression gains are real but modest. The 1.29–1.34× average ratios are substantially below the sub-4-bit compression levels reported for TurboQuant in accelerator-side evaluations.

Figure 1 shows the speed-quality frontier for all 15 configuration–case pairs. No point lies in the upper-right quadrant (faster than baseline and high quality), confirming that the compressed path does not dominate the dense baseline on either dimension in the current implementation.

Figure 1: Speed-quality frontier for all benchmark points. The x-axis is the end-to-end speed ratio relative to the dense baseline (values > 1 would be faster); the y-axis is prefix agreement with the baseline output. The dashed vertical line at $x = 1.0$ marks the dense baseline speed. All 15 points fall below $x = 1.0$. Points in the upper-left cluster are long-context cases (high agreement, severe slowdown); points toward the lower-right are short structured cases (faster but more output divergence).



Speed-quality frontier

6.2 Per-case behavior

Table 2 reports the best observed value for each metric across the three configurations for each prompt case.

Table 2: Per-case best performance across configurations

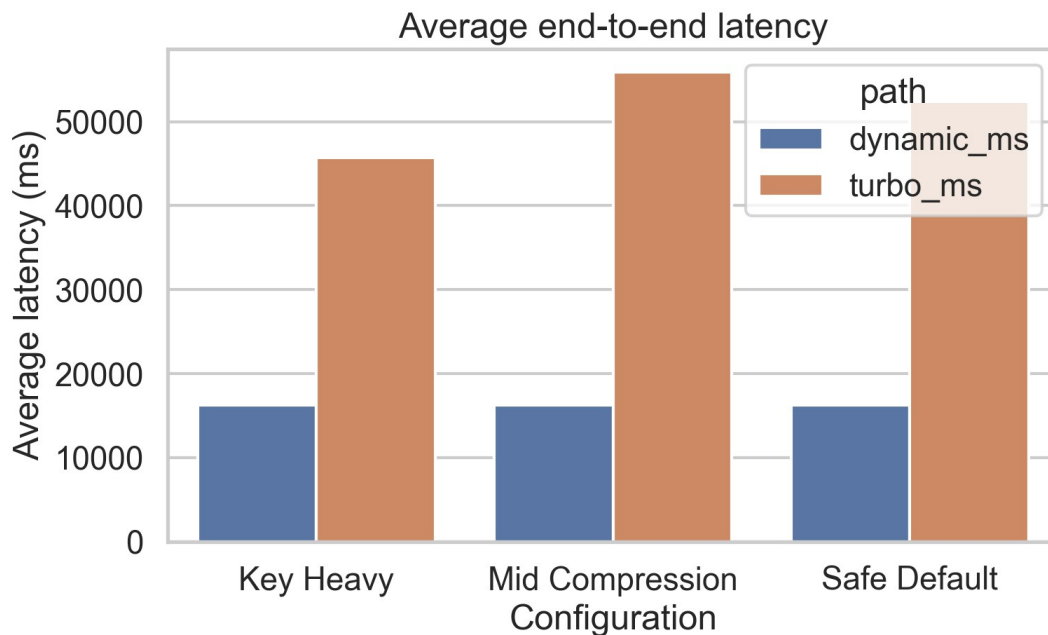
Case	Cache (MB)	Best speed ratio	Best compression	Best prefix agr.
Risk Summary	1.5	0.807× (Key Heavy)	1.18× (Mid)	84.3% (Key Heavy)
Operations Checklist	2.0	0.695× (Key Heavy)	1.25× (Mid)	71.9% (Safe Default)
Policy Comparison	2.2	0.669× (Key Heavy)	1.26× (Mid)	73.2% (Safe Default)
Long Context 1x	23.1	0.370× (Key Heavy)	1.55× (Key Heavy)	93.0% (Key Heavy)
Long Context 2x	43.7	0.323× (Key Heavy)	1.57× (Key Heavy)	96.6% (Safe Default)

Several patterns are visible. First, speed degrades as cache size grows: the best speed ratio for short prompts (0.807×) is over twice the best for long-context prompts (0.323×).

Second, compression improves with cache size: the long-context cases achieve 1.5–1.57× compression while short cases reach only 1.1–1.26×. Third, prefix agreement is high for

long-context cases (92–97%) but much lower and more variable for short structured tasks (36–84%). The worst prefix agreement in the entire sweep is Policy Comparison under Mid Compression (35.8%), which provides concrete evidence against strong quality-preservation claims in the current implementation.

Figure 5 shows the absolute latency comparison across cases and configurations.



Latency comparison

Figure 5: End-to-end generation latency (ms) by case and configuration. Note the dramatic latency increase for the long-context cases under TurboQuant. Long Context 2x takes approximately 171–176 seconds under TurboQuant versus 55 seconds under the dense baseline.

6.3 Decode throughput and TTFT

Tables 3a and 3b report decode throughput (TPS) and TTFT respectively for each case.

Table 3a: Decode throughput (tokens/s) by case and configuration

Case	Dynamic	Safe Default	Key Heavy	Mid Compression
Risk Summary	23.6	18.4	18.5	16.5
Operations Checklist	22.3	15.1	15.2	13.8
Policy Comparison	22.3	14.5	14.6	13.3
Long Context 1x	14.4	1.41	1.53	1.31
Long Context 2x	9.98	0.740	0.737	0.724

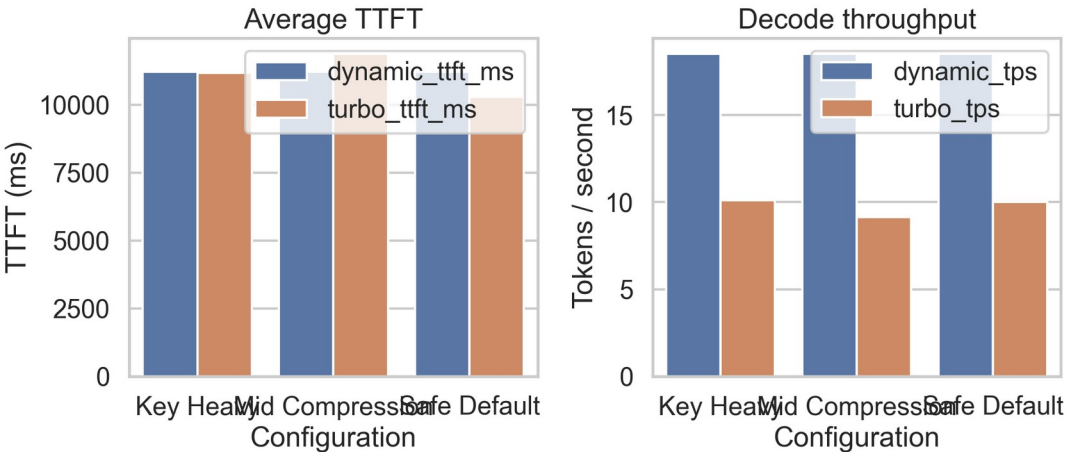
Table 3b: Time-to-first-token (ms) by case and configuration

Case	Dynamic	Safe Default	Key Heavy	Mid Compression
Risk Summary	264	263	261	260
Operations Checklist	258	261	267	265
Policy Comparison	262	268	262	260
Long Context 1x	9,469	8,778	14,132	15,469
Long Context 2x	45,782	41,788	40,952	43,055

The throughput collapse for long-context cases is stark. On Long Context 2x, Safe Default achieves 0.74 tokens/s versus 9.98 tokens/s for the dense baseline — a 13.5× throughput gap. Risk Summary, with its 1.5 MB cache, achieves 18.4 tokens/s versus 23.6 tokens/s for the baseline — a 1.28× gap. This roughly 10× difference in relative throughput between a small and large cache is consistent with per-step reconstruction overhead scaling with cache size (Section 7.2).

TTFT for the three short-context cases (≤ 2.2 MB cache) is nearly identical across all configurations (260–268 ms), as expected: there is no KV state to read before the first decode step, so the cache implementation does not affect TTFT. For the long-context cases, TTFT behavior is more variable. On Long Context 2x, all three TurboQuant configurations have slightly lower TTFT (41–43 seconds) than the dense baseline (45.8 seconds), possibly reflecting reduced initial memory pressure in the compressed path. On Long Context 1x, Safe Default TTFT is slightly lower than the baseline while Mid Compression and Key Heavy are substantially higher. Given the small run budget ($n=2$), we treat these TTFT differences as observations rather than robust findings.

Figure 6 shows TTFT and TPS comparisons across cases and configurations.



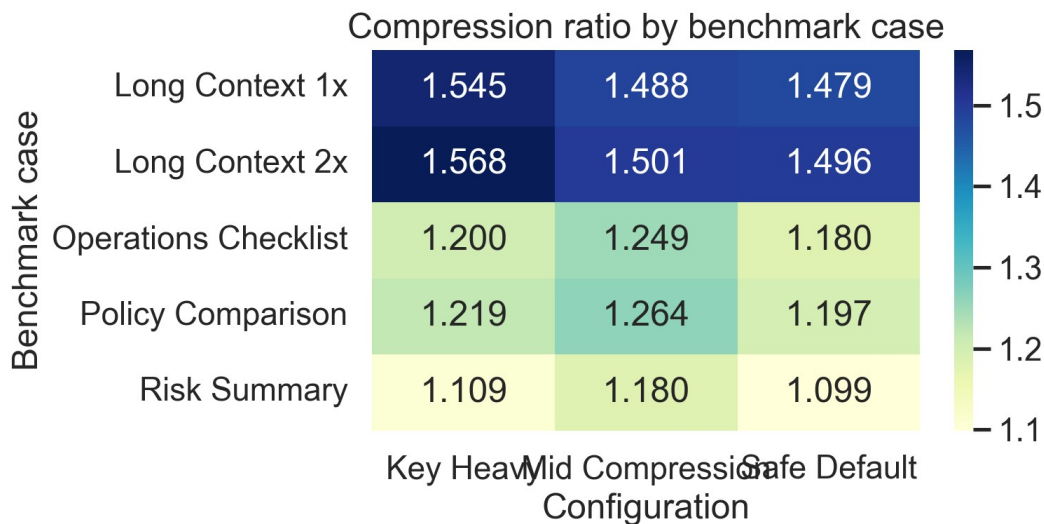
TTFT and TPS comparison

Figure 6: Time-to-first-token (top) and decode throughput (bottom) for all cases and configurations. The throughput collapse for long-context prompts is the dominant visual feature.

6.4 Heatmap summaries

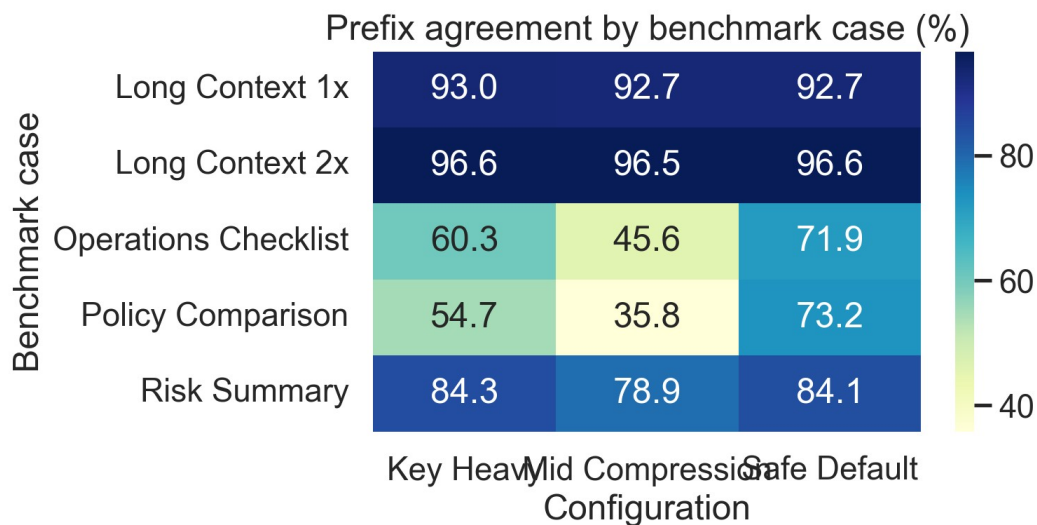
Figures 2–4 show the three primary metrics as heatmaps over the case × configuration matrix.

Figure 2: Compression ratio heatmap (case × configuration). Rows are prompt cases; columns are TurboQuant configurations. Higher compression is observed for long-context cases regardless of configuration.



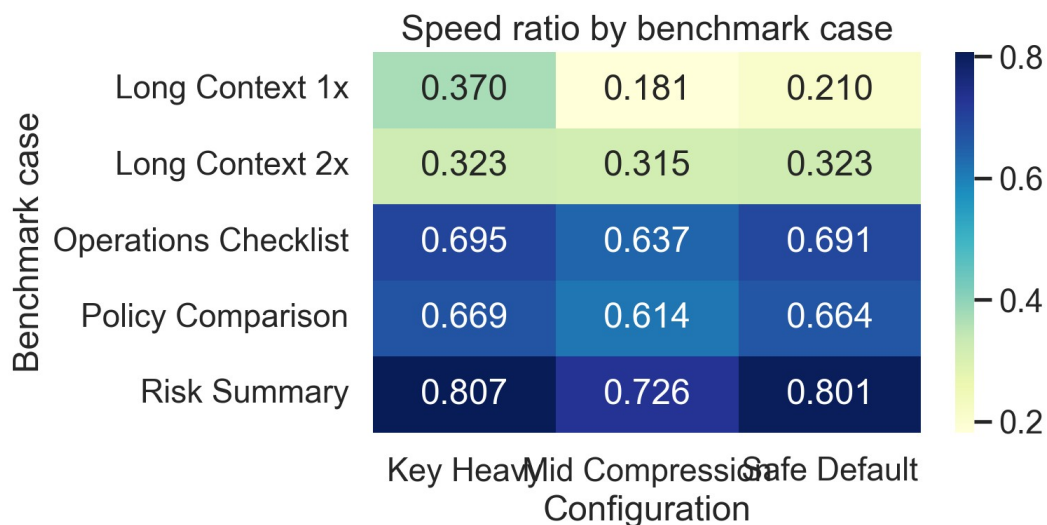
Compression ratio heatmap

Figure 3: Prefix agreement heatmap (case × configuration). Short structured prompts — especially Policy Comparison under Mid Compression — show the worst agreement with the baseline output.



Prefix agreement heatmap

Figure 4: Speed ratio heatmap (case × configuration). All cells are below 1.0 (all configurations are slower than the baseline). The long-context cases are the darkest, reflecting the most severe slowdown.



Speed ratio heatmap

6.5 Configuration-level observations

Safe Default ($b_k=4$, $b_v=8$, $T_r=64$) is the best quality-oriented configuration, with the highest average prefix agreement (83.7%) and a reasonable average speed ratio (0.538×). It should be the reference point for future optimisation work.

Key Heavy ($b_k=3$, $b_v=8$, $T_r=64$) has the best aggregate speed ratio ($0.573\times$) and the highest compression on long-context cases (up to $1.57\times$). It sacrifices some quality (77.8% average prefix agreement) relative to Safe Default.

Mid Compression ($b_k=4$, $b_v=8$, $T_r=48$) is the weakest configuration. It has the worst average speed ratio ($0.495\times$), the worst average prefix agreement (69.9%), and only marginally better average compression ($1.336\times$) than Safe Default ($1.290\times$). Across three of five prompt cases, Mid Compression is simultaneously slower than Safe Default and delivers worse quality — a strictly dominated outcome on those cases. This operating point should not be recommended as a default.

7. Discussion

7.1 Why browser outcomes diverge from accelerator claims

The TurboQuant headline involves three mutually reinforcing components: low-bit compressed storage, efficient attention-kernel consumption of that compressed representation, and accelerator-friendly implementation. The current browser implementation retains only the first: it achieves compressed storage between decode steps. The second and third components are absent. The ONNX decoder consumes standard dense `past_key_values.*` tensors; there is no compressed-attention path, no GPU-resident cache buffer, and no IO-binding that would keep KV state on the GPU across steps.

This is not a critique of TurboQuant as an algorithm. It is a description of what was built and what was not. The browser stack prevents a faithful reproduction not by fundamental impossibility but by API boundary: the ONNX model interface expects dense inputs, and changing that would require modifying the ONNX graph or using ONNX Runtime GenAI-style generation, neither of which is in scope for this study. The result should therefore be read as a report on a cache-wrapper-only integration rather than as evidence about TurboQuant’s intrinsic merit.

7.2 Quantitative evidence for the rematerialization bottleneck

The TPS data in Table 3a provides direct quantitative support for the rematerialization-overhead hypothesis. For the Risk Summary case, the dense KV cache is approximately 1.5 MB. TurboQuant achieves 18.4 tokens/s versus 23.6 tokens/s for the baseline — a $1.28\times$ throughput gap. For Long Context 2x, the dense KV cache is approximately 43.7 MB. TurboQuant achieves 0.74 tokens/s versus 9.98 tokens/s — a $13.5\times$ throughput gap.

If reconstruction overhead were a fixed cost per decode step, the relative throughput gap would be roughly constant across context lengths. Instead, it scales by roughly a factor of 10 as the cache grows from 1.5 MB to 43.7 MB. This scaling is consistent with the materialization step taking time proportional to the number of cached tokens — a natural consequence of allocating, filling, and uploading a dense tensor whose size grows with T at every decode step.

The TTFT pattern provides a complementary observation. For long-context cases, TTFT is at most weakly affected by the compression path (and is slightly lower for some TurboQuant configurations on Long Context 2x). This suggests the performance cost is concentrated in the repeated per-step decode phase rather than in the initial prefill pass, further supporting the per-step rematerialization explanation.

7.3 The configuration landscape

The three tested configurations expose an instructive pattern. Safe Default offers the best quality-speed balance. Key Heavy trades quality for slightly better speed and meaningfully better compression on large caches. Mid Compression, despite having the smallest residual window, is slower than both alternatives and produces worse quality than Safe Default. This combination suggests that the residual window length parameter (T_r) interacts with pack-and-unpack overhead in a non-monotone way: reducing T_r does not linearly reduce reconstruction time but does increase output divergence. Future configuration design should explore this parameter more carefully.

7.4 What the implementation establishes

Despite the negative latency outcome, the implementation contributes several concrete research results.

First, it demonstrates that the transformers.js generation pipeline can support pluggable cache strategies without a total rewrite. The PastKeyValues abstraction is a clean seam for future experimentation.

Second, it shows that browser-based Gemma 4 inference can run a TurboQuant-inspired compressed KV path end-to-end in Chrome WebGPU. Feasibility was not guaranteed.

Third, and most importantly, it identifies the dominant next bottleneck with experimental evidence: the browser stack needs GPU-resident cache handling with reduced or eliminated per-step full-tensor reconstruction, not just a better quantizer. Future work that addresses the system-level issue (Section 9) would start from this evidence rather than from speculation.

8. Limitations and Threats to Validity

Implementation fidelity. The TurboQuantCache is a TurboQuant-inspired approximation. It does not provide a faithful PolarQuant rotation stage or a full QJL residual estimator. Quantization quality, and therefore the compression-quality tradeoff, may differ from a complete implementation. This limits the extent to which results can be attributed to TurboQuant specifically versus to low-bit cache compression in general.

Run budget. Each configuration–case pair was evaluated over 2 runs. With $n=2$, there is no statistical basis for confidence intervals, and results cannot be meaningfully separated from run-to-run variance. The reported means should be treated as point estimates. Some observed differences — particularly the TTFT variations on Long Context 1x — may reflect

noise rather than systematic effects. Future work should use at least 5–10 runs per configuration to establish stability.

Single environment. All experiments were conducted in Chrome WebGPU on a single machine. The hardware GPU model was not recorded, and no cross-browser or cross-GPU benchmarking was performed. Results are specific to this execution environment. Performance on other adapters, browsers, or hardware may differ materially.

Metric scope. Prefix agreement is a lightweight textual proxy for output quality. It captures character-level divergence from the dense-baseline output but does not measure semantic correctness, task-level accuracy, or perplexity. Compressed outputs may score well on prefix agreement while still being practically inferior, or score poorly on prefix agreement while remaining task-useful. The 0/5 exact-match rate confirms that no configuration exactly reproduces the baseline output, but neither exact match nor prefix agreement can substitute for task-specific evaluation.

Quality evaluation is against the dense baseline, not ground truth. The benchmark measures agreement between the compressed path and the dense-cache path. Neither is validated against ground-truth task labels or human evaluation. A quality drop relative to the dense baseline does not necessarily imply poor absolute quality; conversely, high prefix agreement does not guarantee good task performance.

Prompt coverage. Five prompt categories cover a narrow range of text-generation tasks. The suite does not include open-ended generation, multi-turn dialogue, code generation, or quantitative reasoning. Generalization of the quality and speed findings beyond this set is not established.

Lack of hardware specification. The experimental environment was defined by its software configuration (Chrome, WebGPU, q4f16 dtype) but not by a specific GPU. Readers cannot reproduce the absolute latency numbers without the same or equivalent hardware. Relative comparisons between configurations (speed ratios, compression ratios) are more reproducible than absolute timing.

Single model. All experiments use onnx-community/gemma-4-E2B-it-ONNX in q4f16 precision. Results may not generalise to other model families, sizes, or precision levels. The specific KV-cache dimensionalities and layer counts of this model determine the cache sizes reported, which in turn affect the magnitude of the rematerialization overhead.

No comparison with alternative compression methods. The study does not compare the TurboQuant-inspired approach against other KV-cache compression strategies (e.g., token eviction, sliding window, attention sinks) in the browser context. The relative merit of this approach versus alternatives is not established.

9. Future Work

The results in this paper suggest that the most impactful near-term improvements are at the runtime integration level rather than at the quantizer design level.

GPU-resident cache via IO binding. Using ONNX Runtime Web's preferredOutputLocation: 'gpu-buffer' and IO binding to keep KV state on the GPU across decode steps would eliminate or substantially reduce the dominant cost — the CPU-side pack/unpack and the full CPU-to-GPU round trip at each step. This is the single highest-priority system-level change.

Selective rematerialization. Rather than reconstructing the full dense KV cache at every step, future designs could reconstruct only the slice of the cache needed for the current attention window, leaving older segments compressed. This would reduce the materialization cost proportionally to the fraction of the cache accessed per step.

Closer TurboQuant reproduction. A more faithful PolarQuant + QJL implementation may improve the quality-compression tradeoff and would make the browser results more directly comparable to the published accelerator-side claims.

Compressed attention paths. If WebGPU compute shaders can be written to consume compressed KV representations directly — avoiding full reconstruction before the attention operation — the performance model changes qualitatively. This would require co-design with the ONNX graph or a custom attention kernel in WebGPU, which is a larger engineering effort but the path toward genuine speedup.

Statistical robustness. Future benchmark runs should use at least 5–10 runs per configuration to establish variance estimates and support statistical comparisons.

Multi-environment evaluation. Benchmarking across multiple browser vendors (Firefox, Safari), multiple GPU adapter classes (integrated, discrete), and multiple operating systems would establish the generality of the current findings.

Richer quality evaluation. Task-specific metrics — ROUGE scores, F1 over structured outputs, human preference ratings, or perplexity on held-out text — would give a more complete picture of the quality-compression tradeoff than prefix agreement alone.

10. Conclusion

We have presented an experimental browser implementation of a TurboQuant-inspired KV-cache compression path for transformers.js and evaluated it on Gemma 4 in Chrome WebGPU across five prompt categories and three quantizer configurations. The main result is negative on latency and mixed on quality: the compressed path achieves 1.29–1.34× average KV-cache reduction but runs at 0.50–0.57× the end-to-end speed of the dense baseline. Decode throughput drops from 18.5 to approximately 10 tokens/s on average, and collapses to 0.74 tokens/s on the largest tested context. No configuration produces exact-match output.

The practical contribution of this work is not a new state-of-the-art browser inference result. It is a clear experimental baseline that narrows the engineering problem: browser-side KV compression is bottlenecked by per-step dense rematerialization, whose overhead scales with cache size and dominates at long context lengths. Fixing this — by keeping

compressed KV state GPU-resident and eliminating the full reconstruct-on-every-step pattern — is a concrete, achievable next step that the current implementation and benchmark harness are positioned to evaluate.

Data Availability

The benchmark data, derived summary tables, and figures for this study are provided in the paper package:

- `turboquant-benchmark.json` — full benchmark export from the browser harness
- `tables/benchmark_rows.csv` — row-level per-case-per-configuration results
- `tables/config_summary.csv` — aggregate summary by configuration
- `tables/case_summary.csv` — aggregate summary by case
- `figures/` — PNG visualisations generated from the benchmark data

The forked `transformers.js` runtime is available at <https://github.com/stratoslab/transformers.js>. Relevant implementation commits are listed in Section 5.4.

Conflicts of Interest

The authors declare no conflicts of interest.

AI Use Disclosure

AI-assisted tools were used during implementation development and manuscript drafting for this project. All generated content — including code, analysis, and text — was reviewed, edited, and validated by the human authors. The human authors designed the benchmark, ran the experiments, interpreted the results, and take full responsibility for the accuracy of the manuscript and the conclusions drawn.

References

1. Google Research. *TurboQuant: Redefining AI Efficiency with Extreme Compression*. Google Research Blog, 2025. Available at: <https://research.google/blog/turboquant-redefining-ai-efficiency-with-extreme-compression/>
2. *TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate*. OpenReview, 2025. Forum: <https://openreview.net/forum?id=tO3ASKZlok> — Preprint: <https://arxiv.org/pdf/2504.19874>

3. Hugging Face. *Transformers.js: State-of-the-art Machine Learning for the Web*. Documentation. Available at: <https://huggingface.co/docs/transformers.js/index>
4. Hugging Face. *KV Cache Strategies*. Transformers documentation. Available at: https://huggingface.co/docs/transformers/en/kv_cache
5. ONNX Runtime. *Using the WebGPU Execution Provider*. Tutorial documentation. Available at: <https://onnxruntime.ai/docs/tutorials/web/ep-webgpu.html>
6. ONNX Runtime. *Generation AI Configuration Reference*. Available at: <https://onnxruntime.ai/docs/genai/reference/config.html>
7. MDN Web Docs. *WebGPU API*. Mozilla Developer Network. Available at: https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API
8. Preprints.org. *Instructions for Authors*. Available at: <https://www.preprints.org/instructions-for-authors>